

Boletín de Docker Compose

1. Servidor web estático con Nginx

Obxectivo: Servir unha páxina HTML estática mediante Docker Compose.

Tarefas a realizar:

1. Crea un directorio chamado `web-compose` para o exercicio.
2. Dentro do directorio, crea un ficheiro `index.html` co seguinte contido mínimo:
 - Un título principal co texto: **Benvido a Docker Compose!**
 - Un parágrafo adicional explicando que é unha páxina servida por Nginx nun contedor.
3. Crea un ficheiro `docker-compose.yml` que:
 - Defina un servizo chamado `web` baseado na imaxe `nginx:alpine`.
 - Mapee o porto `8080` do host co porto `80` do contedor (é dicir, `8080:80`).
 - Monte o ficheiro `index.html` local na ruta do contedor `/usr/share/nginx/html/index.html`.
 - Opcional: establece un `container_name` identificable (ex. `web_nginx`).
4. Arrinca o proxecto en **modo detached**.
5. Proba no navegador en `http://localhost:8080` e comproba que se ve a páxina.
6. Pecha e limpa os recursos creados ao rematar.

Entrega esperada:

- Estrutura de directorios do exercicio.
- O ficheiro `docker-compose.yml`.
- O ficheiro `index.html`.

2. Aplicación Flask con base de datos MySQL

Obxectivo: Levantar unha aplicación web Python (Flask) que se conecte a unha base de datos MySQL, todo orquestrado con Docker Compose.

Tarefas a realizar:

1. Crea un directorio chamado `flask-mysql`.
2. Dentro, crea un subdirectorio `app` para a aplicación Flask.
3. Na carpeta `app`, crea estes ficheiros:
 - `app.py`: Unha rota raíz `/` que devolva un texto lido a través dunha consulta á BD. Debe conectarse a MySQL usando as variables de contorno (contrasinal, host, etc.).
 - `requirements.txt`: Inclúe polo menos `Flask==3.0.0` e `mysql-connector-python==9.1.0`.
 - `Dockerfile`: Use `python:3.12-slim`, instale dependencias de `requirements.txt` e execute `python app.py`.
4. No nivel superior, crea `docker-compose.yml` con:
 - Servizo `web` que **se constrúe** dende `./app`, expón `5000:5000`, e depende do servizo `db`.
 - Servizo `db` baseado en `mysql:8.0` que estableza:
 - `MYSQL_ROOT_PASSWORD` (ex. `1234`)
 - `MYSQL_DATABASE` (ex. `demo`)

- Un volume persistente montado en `/var/lib/mysql`.
- 5. Arrinca en **modo detached** e comproba en `http://localhost:5000` que a app responde cunha mensaxe (por exemplo: *Docker Compose con Flask e MySQL funcionando!*).
- 6. Pecha e **limpa tamén o volume** ao finalizar (para deixar todo como estaba).

Entrega esperada:

- Estrutura de directorios completa do exercicio.
- Contido de `docker-compose.yml`.
- Contido de `app/app.py`, `app/requirements.txt` e `app/Dockerfile`.